

GROWTHLIFT DIGITAL

Week 3 Guide

React.js — Components, Hooks & State

Self-paced · Remote · 5 days of tasks, tutorials & projects

JSX & Setup

Props & State

Hooks & Lists

React Router

Tailwind + Deploy

YOUR 5-DAY JOURNEY

DAY 1

JSX & Setup

DAY 2

Props & State

DAY 3

Hooks & Lists

DAY 4

React Router

DAY 5

Tailwind & Deploy

WEEK 3 DELIVERABLES

- ◆ Vite + React project scaffolded with organised component folder structure
- ◆ Reusable Card, Counter, Toggle, and QuoteBox components built from scratch
- ◆ Multi-page React app using React Router v6 — Home, About, Projects, Contact, 404
- ◆ NavLink with active class — professional navigation with visual feedback
- ◆ Full app styled with Tailwind CSS — responsive across mobile, tablet, desktop
- ◆ Live React app deployed on Vercel with a public URL

PREREQUISITE — COMPLETE BEFORE STARTING

You must have Node.js and npm installed (Week 1 setup). Confirm by running `node -v` in terminal.
Your Week 1 and 2 work should be on GitHub. Week 3 is a fresh project (`npm create vite`).

FOUNDER

Abdul Rehman

0328 0055453

CTO

Asad Riaz

growthliftpk@gmail.com

WEBSITE

GrowthLift

growthlift-digital.vercel.app

DAY 1

JSX & Setup

DAY 2

Props & State

DAY 3

Hooks & Lists

DAY 4

React Router

DAY 5

Tailwind & Deploy

React is a JavaScript library for building user interfaces with reusable components. Today you will scaffold your first React project and understand how JSX works.

YOUR TO-DO LIST — DAY 1

☐ Make sure Node.js is installed — open terminal and run: **node -v** and **npm -v**. Both should show version numbers

☐ Create your first React project using Vite (faster than Create React App):

→ Open terminal in your projects folder and run:

→ **npm create vite@latest growthlift-week3 -- --template react**

→ **cd growthlift-week3**

→ **npm install** (installs all dependencies — wait for it to finish)

→ **npm run dev** (starts the dev server)

☐ Open your browser at **http://localhost:5173** — you should see the Vite + React welcome page

☐ Open the project in VS Code: **code .** in the terminal

☐ EXPLORE THE FOLDER STRUCTURE — understand what each folder/file does:

→ **src/** — all your React code lives here

→ **src/main.jsx** — entry point, mounts the App into the HTML

→ **src/App.jsx** — the root component of your app

→ **public/** — static assets (images, fonts) that don't get processed

→ **index.html** — the single HTML file. React injects everything into `<div id="root">`

→ **vite.config.js** — build configuration. No need to change this yet

☐ Open **src/App.jsx** and delete everything inside it. Replace with this clean version:

→ **function App() { return <h1>Hello GrowthLift!</h1>; }**

→ **export default App;**

☐ Watch the browser update automatically — this is called **Hot Module Replacement (HMR)**

☐ UNDERSTAND JSX — it looks like HTML but it is JavaScript. Key differences:

→ Use **className** instead of class (class is a JS keyword)

→ Every JSX element must have one root element — wrap in `<div>` or `<>` (fragment)

→ Self-close empty tags: `<img />` `<input />` `<br />`

→ JavaScript expressions go inside curly braces: `{2 + 2}` or `{userName}`

☐ Create your first custom component — add a new file **src/components/Header.jsx**:

→ Write: `function Header() { return <header><h1>GrowthLift Intern</h1></header>; }`

→ Add: `export default Header;`

☐ Import and use Header inside App.jsx: `import Header from "../components/Header";`

☐ Replace your h1 in App.jsx with `<Header />` — see it render in the browser

☐ Commit: `git add . && git commit -m "Day 11: React setup, Vite, first component"`

☐ Post a screenshot of your browser showing your React app: "Day 11 Done ■"

LEARNING RESOURCES — DAY 1

YT

React JS Crash Course

youtube.com/watch?v=w7eJDZ8Swv8

Traversy Media — best React beginner tutorial, 90 min

VT

Vite + React Setup

youtube.com/watch?v=MdMQcRJjVuk

Kevin Powell — setting up Vite with React

RC

React Official Docs

react.dev/learn

New React docs — interactive and very beginner friendly

JS

JSX in Depth

react.dev/learn/writing-markup-with-jsx

Official explanation of JSX rules and syntax

MENTOR TIP — REACT MENTAL MODEL

Stop thinking in pages. In React, you think in **components** — reusable building blocks.

Every UI element (Header, Card, Button, Footer) is its own component. Build small, compose big.

JSX is NOT HTML. It compiles to JavaScript. Always use **className** not class, and close every tag.

WHAT YOU LEARNED TODAY

- Scaffolded a React + Vite project — fastest modern React setup
- Understood the folder structure: src/, components/, main.jsx, App.jsx
- Learned JSX rules: className, fragments, self-closing tags, JS expressions in {}
- Created your first custom component and imported it into App

DAY 1

JSX & Setup

DAY 2

Props & State

DAY 3

Hooks & Lists

DAY 4

React Router

DAY 5

Tailwind & Deploy

Props let you pass data into components. `useState` lets components remember and update data. These are the two most fundamental React concepts — everything else builds on them.

YOUR TO-DO LIST — DAY 2

☐ Open your Week 3 project and run: `npm run dev`☐ PROPS — passing data from parent to child component:→ Create `src/components/Card.jsx`→ Write:

```
function Card({ title, description, color }) { return ( <div style={{ borderLeft: `4px solid ${color}` }}>  
<h3>{title}</h3> <p>{description}</p> </div> ); }
```

→ Export it: `export default Card;`→ In `App.jsx`, import `Card` and use it 3 times with different props:→ `<Card title="React" description="JS library for UIs" color="#A78BFA" />`→ `<Card title="Vite" description="Fast build tool" color="#00C864" />`→ `<Card title="Tailwind" description="Utility-first CSS" color="#00B4D8" />`☐ Notice: same component, different data — this is the power of props☐ USESTATE HOOK — making components interactive:→ Create `src/components/Counter.jsx`→ At the top: `import { useState } from "react";`→ Inside the function: `const [count, setCount] = useState(0);`→ Return: `<div> <p>Count: {count}</p> <button onClick={() => setCount(count + 1)}>+1</button> <button
onClick={() => setCount(count - 1)}>-1</button> <button onClick={() => setCount(0)}>Reset</button> </div>`→ Import `Counter` in `App.jsx` and add `<Counter />` below your cards☐ PRACTICE — build a toggle component:→ Create `src/components/Toggle.jsx`

→ Use: `const [isVisible, setIsVisible] = useState(false);`

→ Button that says "Show" when hidden, "Hide" when showing

→ Below the button: `{isVisible && <p>This content is now visible!</p>}` — this is conditional rendering

☐ UNDERSTAND the React re-render rule: when state changes, React re-renders that component

☐ Never modify state directly: `count = count + 1` is WRONG. Always use `setCount()`

☐ Commit: `git add . && git commit -m "Day 12: Props, useState, conditional rendering"`

☐ Post screenshots of your Counter and Toggle working: "Day 12 Done ■"

LEARNING RESOURCES — DAY 2



React Props Guide

react.dev/learn/passing-props-to-a-component

Official React docs on props — clear with examples



useState Hook

react.dev/reference/react/useState

Full useState reference with patterns and examples



Props and State

youtube.com/watch?v=IYvD9oBCuJI

Codevolution — 20 min deep dive into props and state



Conditional Rendering

youtube.com/watch?v=7o5FPaVA9m0

Web Dev Simplified — all ways to conditionally render

MENTOR TIP — STATE RULES

Rule 1: Never mutate state directly. Always call the setter function: `setCount(newValue)`.

Rule 2: State is local to the component. If two components need the same state, lift it to their parent.

Rule 3: When you call a setter, React schedules a re-render — it does not update immediately in the same line.

WHAT YOU LEARNED TODAY

→ Understood props — passing data from parent to child, making components reusable

→ Used useState to build a Counter with increment, decrement, and reset

→ Built a Toggle component using conditional rendering with the `&&` operator

→ Learned the fundamental rule: never mutate state directly, always use the setter

DAY 1

JSX & Setup

DAY 2

Props & State

DAY 3

Hooks & Lists

DAY 4

React Router

DAY 5

Tailwind & Deploy

useEffect runs code after a component renders — perfect for fetching data or syncing with the browser. Today you also learn to render lists properly and understand why React needs the "key" prop.

YOUR TO-DO LIST — DAY 3

☐ Open your project and run: **npm run dev**☐ USEEFFECT — the side effects hook:

→ Add to Counter.jsx (or any component): **import { useState, useEffect } from "react";**

→ Inside the component, add: **useEffect(() => { document.title = `Count: \${count}`; }, [count]);**

→ Watch your browser tab title update every time you click the counter buttons

→ The array at the end — **[count]** — is the dependency array. Effect runs when count changes

☐ UNDERSTAND THE 3 FORMS of useEffect:

→ **useEffect(() => { ... });** — runs after EVERY render (no array)

→ **useEffect(() => { ... }, []);** — runs ONCE after first render (empty array)

→ **useEffect(() => { ... }, [value]);** — runs when "value" changes

☐ FETCH DATA WITH USEEFFECT — create **src/components/QuoteBox.jsx**:

→ **const [quote, setQuote] = useState("");**

→ **const [loading, setLoading] = useState(true);**

→ **useEffect(() => { fetch("https://api.quotable.io/random") .then(r => r.json()) .then(data => { setQuote(data.content); setLoading(false); }); }, []);**

→ Return: **{loading ? <p>Loading...</p> : <p>"{quote}"</p>}**

→ Add **<QuoteBox />** to App.jsx and see it fetch and display a quote on load

☐ RENDERING LISTS — create **src/components/SkillsList.jsx**:

→ **const skills = ["HTML", "CSS", "JavaScript", "React", "Node.js"];**

→ Return: `<ul> {skills.map((skill, index) => ( <li key={index}>{skill}</li> ))} </ul>`

→ The **key** prop is required when rendering lists — React uses it to track which item is which

☐ PRACTICE — render a list of intern card objects:

→ Create an array of objects: `[[ { id: 1, name: "Ali", city: "Lahore" }, { id: 2, name: "Sara", city: "Karachi" } ]]`

→ Map over it and render a Card component for each, passing name and city as props

→ Use `key={intern.id}` — always prefer a unique ID over index as the key

☐ Commit: `git add . && git commit -m "Day 13: useEffect, data fetching, lists and keys"`

☐ Post screenshots of QuoteBox fetching data and your list rendering: "Day 13 Done 🎉"

LEARNING RESOURCES — DAY 3

RC

useEffect Hook

react.dev/reference/react/useEffect

Official React useEffect docs with all patterns

YT

useEffect Explained

youtube.com/watch?v=0ZJgIjIuY7U

Web Dev Simplified — best useEffect explanation

RC

Rendering Lists

react.dev/learn/rendering-lists

React official guide on lists and keys

YT

Fetching Data in React

youtube.com/watch?v=001xm_doFYw

Traversy Media — fetching API data with useEffect

MENTOR TIP — USEEFFECT MISTAKES

Forgetting the dependency array causes an **infinite loop** — the effect fires, updates state, triggers re-render, fires again. Never fetch data inside the render — always inside useEffect with an empty dependency array `[]`. If you see "Warning: Can't perform state update on unmounted component" — you need a cleanup function in useEffect.

WHAT YOU LEARNED TODAY

- Used useEffect to update the browser tab title reactively when state changes
- Fetched live API data inside useEffect with a loading state — professional pattern
- Rendered lists using `.map()` with the required key prop
- Learned the 3 forms of useEffect and when to use each one

DAY 1

JSX & Setup

DAY 2

Props & State

DAY 3

Hooks & Lists

DAY 4

React Router

DAY 5

Tailwind & Deploy

React apps are Single Page Applications (SPAs) — only one HTML file, but multiple "pages".
React Router handles navigation between these pages without reloading the browser.

YOUR TO-DO LIST — DAY 4

- ☐ Open your project terminal and install React Router: **npm install react-router-dom**
- ☐ After install, verify it's in package.json under "dependencies"
- ☐ CREATE YOUR PAGE COMPONENTS — add these files inside **src/pages/**:
 - **src/pages/Home.jsx** — return a heading "Home Page" and a short paragraph
 - **src/pages/About.jsx** — return "About Me" page with your name and a bio
 - **src/pages/Projects.jsx** — return "My Projects" with 3 project card placeholders
 - **src/pages/Contact.jsx** — return "Contact" with name, email, message fields (just HTML, no JS yet)
- ☐ SETUP ROUTER IN main.jsx:
 - **import { BrowserRouter } from "react-router-dom";**
 - Wrap the **<App />** in **<BrowserRouter>**: **<BrowserRouter><App /></BrowserRouter>**
- ☐ SETUP ROUTES IN App.jsx:
 - **import { Routes, Route } from "react-router-dom";**
 - Import all 4 page components at the top of App.jsx
 - Replace your App return with: **<Routes> <Route path="/" element={<Home />} /> <Route path="/about" element={<About />} /> <Route path="/projects" element={<Projects />} /> <Route path="/contact" element={<Contact />} /> </Routes>**
- ☐ UPDATE YOUR NAVBAR to use React Router links:
 - In Header.jsx: **import { Link } from "react-router-dom";**
 - Replace **** with **<Link to="/about">About</Link>**
 - NEVER use **<a href>** for internal navigation in React — it causes a full page reload

☐ ADD AN ACTIVE LINK STYLE — use NavLink instead of Link:

→ `import { NavLink } from "react-router-dom";`

→ `<NavLink to="/about" className={({ isActive }) => isActive ? "active-link" : ""}>About</NavLink>`

→ Add `.active-link` style in your CSS: different color or underline

☐ ADD A 404 PAGE — create `src/pages/NotFound.jsx`:

→ Return: `<h1>404 — Page Not Found</h1>`

→ Add to your Routes: `<Route path="*" element={ <NotFound /> } />`

☐ Test by visiting `http://localhost:5173/anything-wrong` — should show 404 page

☐ Commit: `git add . && git commit -m "Day 14: React Router, pages, NavLink, 404"`

☐ Post screenshots of clicking between pages: "Day 14 Done ■"

LEARNING RESOURCES — DAY 4

RR

React Router Docs

reactrouter.com/en/main

Official React Router v6 documentation

YT

React Router v6

youtube.com/watch?v=U13y1LXxzdu

Web Dev Simplified — complete React Router tutorial

YT

SPA Explained

youtube.com/watch?v=w1Q5gV38bF8

Fireship — what is a Single Page Application?

RC

NavLink API

reactrouter.com/en/main/components/nav-link

NavLink with active class styling docs

MENTOR TIP — ROUTING BEST PRACTICES

Always use `<Link>` or `<NavLink>` for internal navigation — never `<a href>`. It prevents full page reloads.

Keep page components in `src/pages/` and reusable UI pieces in `src/components/`. Consistent structure matters.

The `path="*" wildcard route always comes last — React Router tries routes top to bottom.`

WHAT YOU LEARNED TODAY

- Installed React Router and set up BrowserRouter, Routes, and Route
- Built 4 page components: Home, About, Projects, Contact
- Used NavLink with dynamic active class — professional navigation pattern
- Added a 404 catch-all route with the path="*" wildcard

DAY 1

JSX & Setup

DAY 2

Props & State

DAY 3

Hooks & Lists

DAY 4

React Router

DAY 5

Tailwind & Deploy

Tailwind CSS replaces writing custom CSS with utility classes directly in your JSX. Today you style your whole React app with Tailwind and deploy it live on Vercel.

YOUR TO-DO LIST — DAY 5

☐ Install Tailwind CSS in your Vite + React project:

→ Run: **npm install -D tailwindcss postcss autoprefixer**

→ Run: **npx tailwindcss init -p** (creates tailwind.config.js and postcss.config.js)

☐ CONFIGURE TAILWIND — open tailwind.config.js and set the content array:

→ **content: [".index.html", "src/**/*.js,ts,jsx,tsx"]** — tells Tailwind which files to scan

☐ SETUP TAILWIND IN CSS — open **src/index.css** and replace everything with:

→ **@tailwind base;**

→ **@tailwind components;**

→ **@tailwind utilities;**

☐ Restart the dev server: stop with Ctrl+C, then run **npm run dev** again☐ STYLE YOUR HOME PAGE with Tailwind utility classes:

→ Add to your main heading: **className="text-4xl font-bold text-white mb-4"**

→ Wrap your page in: **className="min-h-screen bg-gray-900 text-white p-8"**

→ Style a card: **className="bg-gray-800 rounded-xl p-6 border border-gray-700 hover:border-green-400 transition-colors"**

☐ STYLE YOUR NAVBAR:

→ Header: **className="flex items-center justify-between px-8 py-4 bg-gray-900 border-b border-gray-800"**

→ Nav links: **className="text-gray-400 hover:text-green-400 transition-colors font-medium"**

→ Active link: **className="text-green-400 border-b-2 border-green-400"**

☐ MAKE IT RESPONSIVE with Tailwind breakpoints:

→ `className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6"` — responsive card grid

→ `className="text-2xl md:text-4xl lg:text-6xl"` — responsive font sizes

☐ DEPLOY TO VERCEL:

→ Run: **npm run build** — creates a production-ready dist/ folder

→ Create a free account at **vercel.com**

→ Click "New Project" → "Import Git Repository" → connect your GitHub

→ Select your growthlift-week3 repo → click Deploy

→ Wait 1–2 minutes — Vercel auto-detects Vite and builds it correctly

→ Copy your live URL: **yourusername-growthlift-week3.vercel.app**

☐ Post your live Vercel URL in the community: "Week 3 Complete! React app is LIVE ■"☐ Commit everything: **git add . && git commit -m "Day 15: Tailwind CSS + deployed to Vercel"**

LEARNING RESOURCES — DAY 5

**Tailwind Docs**tailwindcss.com/docs

Official Tailwind CSS documentation — searchable

**Tailwind + Vite Setup**tailwindcss.com/docs/guides/vite

Official step-by-step Vite integration guide

**Vercel Deploy Guide**vercel.com/docs/deployments/overview

How Vercel builds and deploys your project

**Tailwind Crash Course**youtube.com/watch?v=dFgzH0X84xQ

Traversy Media — Tailwind CSS from scratch

MENTOR TIP — TAILWIND IN REAL PROJECTS

Tailwind looks messy at first. After 2–3 days it becomes faster than writing CSS by hand.

Use the **Tailwind CSS IntelliSense** VS Code extension — it autocompletes class names as you type.

GrowthLift Digital uses Tailwind for all React client projects. Mastering it this week means you are production-ready.

WHAT YOU LEARNED TODAY

- Installed and configured Tailwind CSS in a Vite + React project
- Styled a full multi-page app using only utility classes — no custom CSS files
- Used responsive breakpoints: `grid-cols-1 md:grid-cols-2 lg:grid-cols-3`
- Deployed a live React app to Vercel — it is on the internet right now

React.js Cheatsheet

JSX RULES & PATTERNS

<code>className="..."</code>	Use className, not class (class is a ...)
<code>{expression}</code>	Embed any JS expression inside JSX
<code><Comp /></code>	Self-closing for components with no c...
<code><> ... </></code>	Fragment — wrap multiple elements, ...
<code>{isTrue && <p/>}</code>	Conditional render — shows only wh...
<code>{flag ? <A/> : }</code>	Ternary — show A or B based on flag
<code>key={item.id}</code>	Required unique key when rendering ...
<code>style={{ color: "red" }}</code>	Inline styles use camelCase, double c...

REACT ROUTER v6

<code><BrowserRouter></code>	Wrap your app — enables routing
<code><Routes> <Route .../> </R...></code>	Define all your app routes
<code><Route path="/p" element=...</code>	Map URL path to a component
<code><Route path="*" element={...</code>	Catch-all for unknown URLs
<code><Link to="/about"></code>	Navigate without page reload
<code><NavLink to="/" className=...</code>	Link with active class support
<code>useNavigate()</code>	Hook to navigate programmatically
<code>useParams()</code>	Hook to read URL params like /user/i...

REACT HOOKS

<code>const [val, setVal] = use...</code>	State variable with initial value x
<code>setVal(newValue)</code>	Update state — triggers re-render
<code>useEffect(() => {...}, [])...</code>	Run once after first render (empty de...
<code>useEffect(() => {...}, [v...</code>	Run when val changes
<code>const ref = useRef(null)</code>	Reference to a DOM element
<code>ref.current.focus()</code>	Access the actual DOM element via r...
<code>useCallback(() => fn, [de...</code>	Memoize a function to avoid re-creati...
<code>useMemo(() => compute, [d...</code>	Memoize an expensive computed val...

TAILWIND CSS

<code>text-xl font-bold text-wh...</code>	Large bold white text
<code>bg-gray-900 min-h-screen</code>	Dark full-height background
<code>flex items-center justify...</code>	Flexbox centered + space between
<code>grid grid-cols-3 gap-6</code>	CSS Grid — 3 columns with gap
<code>md:grid-cols-2 lg:grid-co...</code>	Responsive columns at breakpoints
<code>rounded-xl border border-...</code>	Rounded card with border
<code>hover:text-green-400 tran...</code>	Color change on hover with animatio...
<code>p-6 px-8 py-4</code>	Padding: all / horizontal / vertical

QUESTIONS? GET STUCK? REACH OUT.

WhatsApp Community · growthliftpk@gmail.com · 0328 0055453

Abdul Rehman (Founder) · Asad Riaz (CTO) · growthlift-digital.vercel.app